HTTP의 문제점과 HTTPS

왜 HTTP로 충분하지 않고, HTTPS 가 필요할까?

우아한테크코스 8기 BE 도우너

목차

1. HTTP의 특징과 문제점
2. 암호화 방식 - 대칭키, 비대칭키
3. 인증서와 CA
4. HTTPS 통신 과정

HTTP의 특징



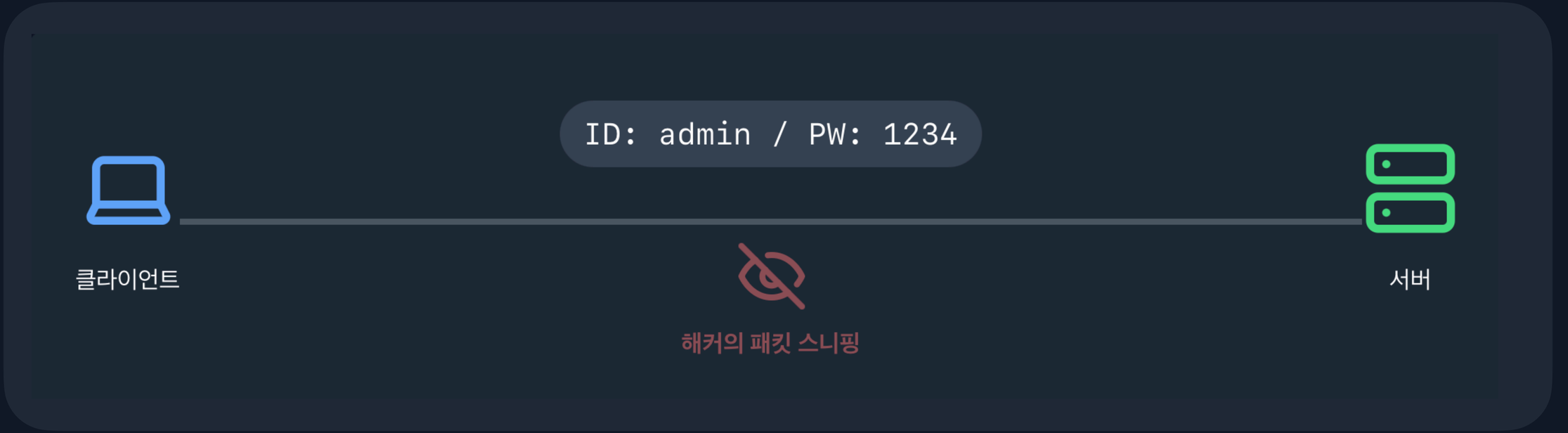
HTTP (HyperText Transfer Protocol)

웹에서 서버와 클라이언트가 데이터를 주고받는 기본 규칙

치명적 약점 : '평문' 통신

데이터가 암호화되지 않아 누구나 읽을 수 있음

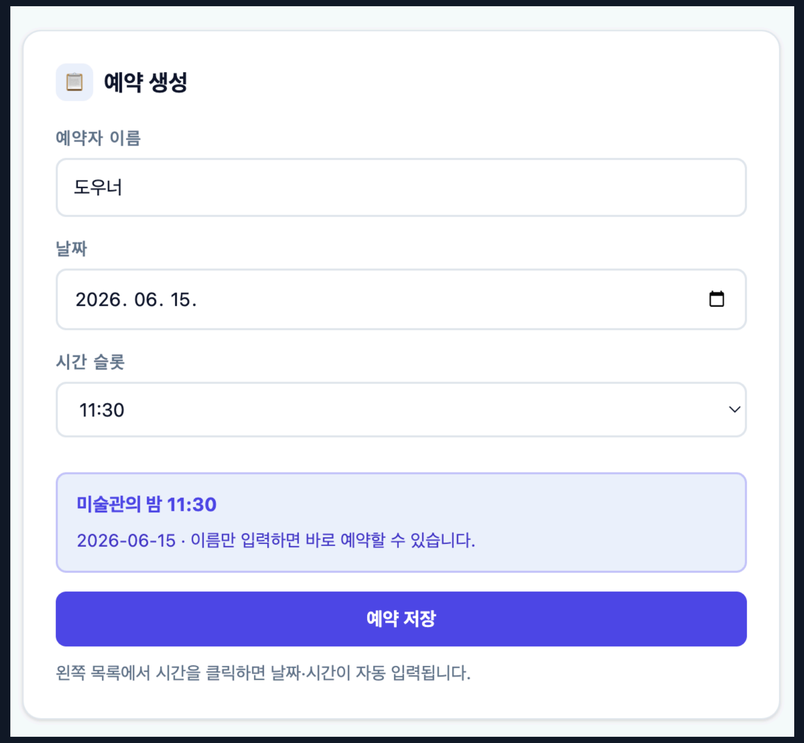
HTTP의 문제점 - 도청

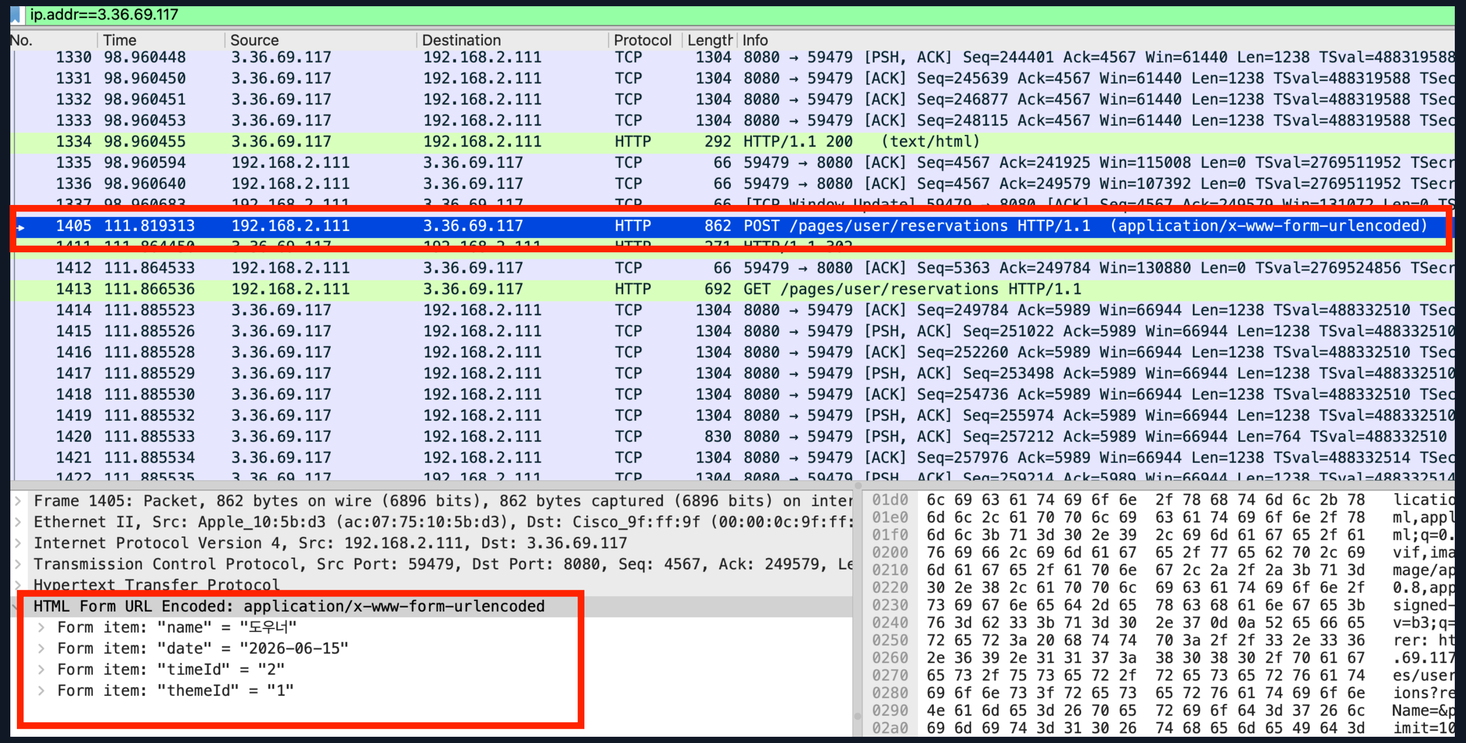


암호화되지 않은 데이터는 중간에서 그대로 들여다 볼 수 있음

-> 개인정보 및 비밀번호 유출 위험

HTTP의 문제점 - 도청



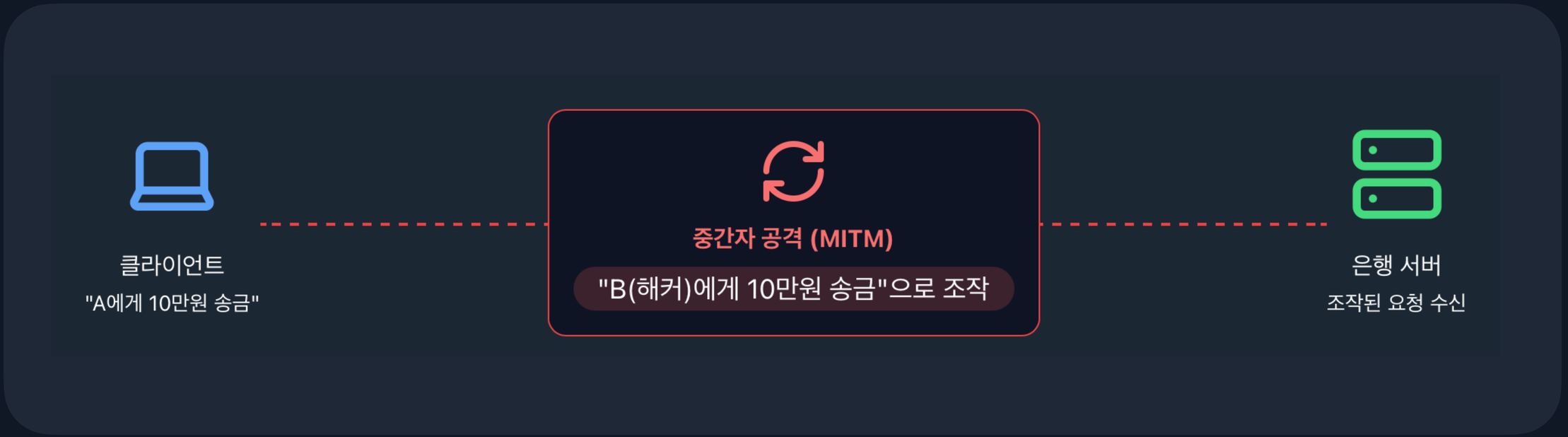


방탈출 예약 서비스에서 예약 저장 요청
(Ec2 서버, HTTP)

→

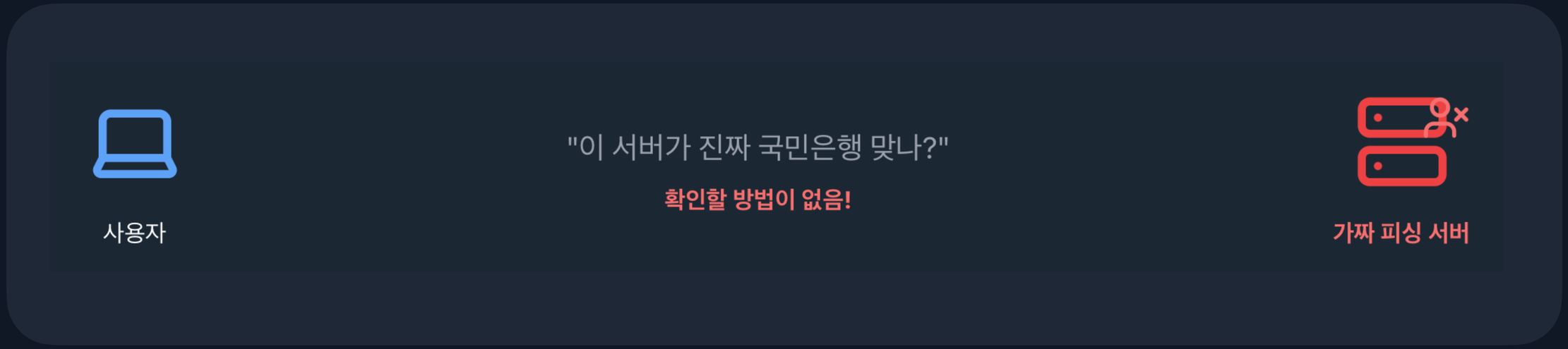
Wireshark에서 패킷 캡쳐 시 요청한 폼 데이터를 그대로 확인할 수 있음

HTTP의 문제점 - 변조



데이터가 중간에 변경되었는지 알 수 없음

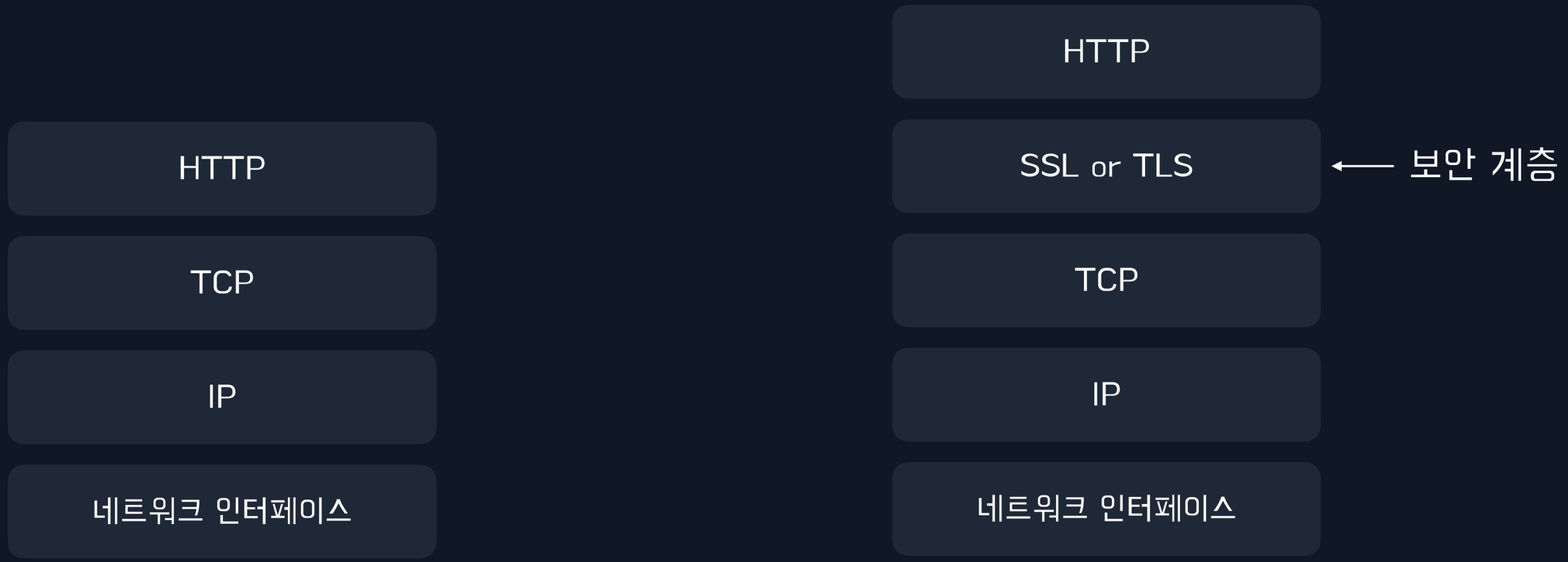
HTTP의 문제점 - 위장



통신 상대방의 신원을 확인하는 절차가 없음

-> 가짜 사이트에 접속하여 정보 탈취당할 위험

HTTPS


HTTP
TCP
IP
네트워크 인터페이스
HTTP
SSL or TLS
보안 계층
TCP
IP
네트워크 인터페이스
HTTPS

암호화 방식 - 대칭키 방식

암호화와 복호화에 **동일한 1개의 키**를 사용



장점

연산 속도가 빠름

단점

키를 전달할 때 중간에서 가로챌 수 있음

암호화 방식 - 공개키 (비대칭키) 방식

암호화와 복호화에 서로 다른 2개의 키를 사용



장점

중간에 데이터를 가로채도, '개인키'가 없어
해독 불가

단점

암호화, 복호화를 위해 CPU나 메모리 등의
하드웨어 리소스 소모가 필요하며 통신 속도가 느림

HTTPS = 하이브리드 암호화

- 대칭키와 공개키의 장점을 활용

- 초기 통신 연결 시 공개키(비대칭키) 방식 = 안정성

- 실제 데이터 전송 시 대칭키 방식 = 속도

HTTPS = 하이브리드 암호화

- 대칭키와 공개키의 장점을 활용

- 초기 통신 연결 시 공개키(비대칭키) 방식 = 안정성

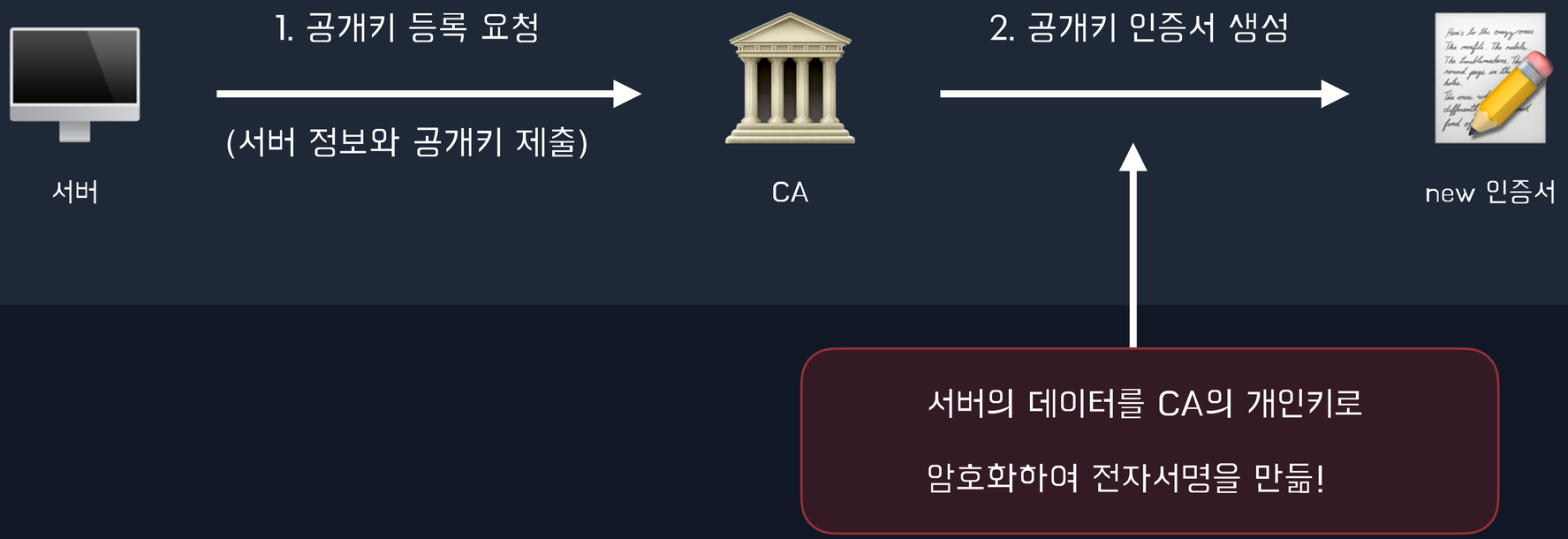
- 실제 데이터 전송 시 대칭키 방식 = 속도

하이브리드 암호화를 사용한다 해도

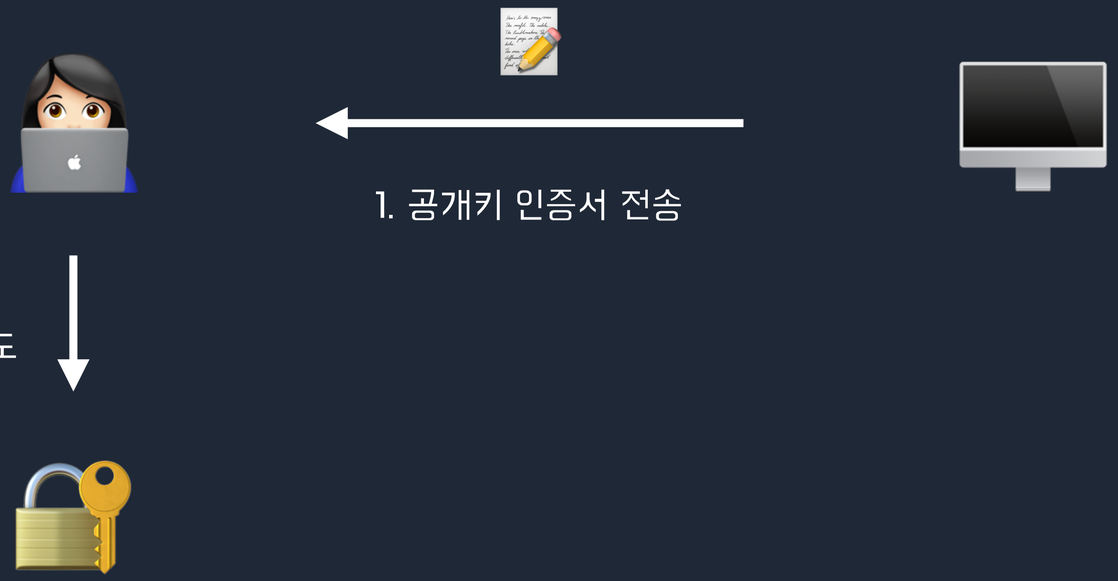
서버의 위장은 보장할 수 없음!

인증서와 CA

- '위장' 문제를 막기 위한 서버의 디지털 신분증
- 인증 기관 (CA)과 이 기관이 발행하는 공개키 증명서 이용



인증서와 CA | 변조 확인 방식



복호화 성공

통신하려던 서버

복호화 실패

위조된 서버

HTTPS 통신 과정





Client Hello (암호화 방식 제안)

Server Hello (암호화 방식 선택)

Certificate 서버 인증서 전송

Server Hello Done

브라우저에 저장되어 있는
CA의 공개키로 인증서 복호화

인증서 검증

인증서 있던 서버의
공개키로 대칭키 암호화

Client Key Exchange

Change Cipher Spec

Finish

개인키로 전송 받은
공개키 복호화

Change Cipher Spec

Finish

여기서부터는 대칭키로 패킷 주고 받음

HTTPS 통신 과정





신뢰할 수 있는 서버인지 + 서버의 공개키 얻기

Client Hello (암호화 방식 제안)

Server Hello (암호화 방식 선택)

Certificate 서버 인증서 전송

Server Hello Done

브라우저에 저장되어 있는
CA의 공개키로 인증서 복호화

인증서 검증

서버 공개키로 대칭키 주고 받기

인증서 있던 서버의
공개키로 대칭키 암호화

Client Key Exchange

Change Cipher Spec

Finish

개인키로 전송 받은
공개키 복호화

Change Cipher Spec

Finish

여기서부터는 공개키로 패킷 주고 받음

HTTPS 통신 과정





Client Hello (암호화 방식 제안)

Server Hello (암호화 방식 선택)

Certificate 서버 인증서 전송

Server Hello Done

브라우저에 저장되어 있는
CA의 공개키로 인증서 복호화

인증서 검증

인증서 있던 서버의
공개키로 대칭키 암호화

Client Key Exchange

Change Cipher Spec

Finish

개인키로 전송 받은
공개키 복호화

Change Cipher Spec

Finish

공개키(비대칭키) 암호화 - 보안

대칭키 암호화 - 속도

여기서부터는 공개키로 패킷 주고 받음

결론

- HTTP는 평문 통신으로 인해 도청, 변조, 위장 문제가 존재
- HTTPS는 SSL/TLS를 이용해 데이터를 암호화
  - 공개키 암호화와 대칭키 암호화를 함께 사용하는 하이브리드 방식을 사용
  - 인증서와 CA를 통해 서버의 신뢰성을 검증

